Functional dependency

A functional dependency (FD) is a relationship between attributes in a database where the value of one attribute (or set of attributes) uniquely determines the value of another attribute. It is represented by →

Notation:

If A → B,

Here A is determinant
B is dependent

A determines B (left to right)

B is functionally dependent on A. (right to left)

Student_ID	Name	Department
101	Ram	Computer Science
102	Amol	Mechanical
103	Priya	Computer Science

Example: Consider a STUDENT table:

Here:

Student_ID → Name

This means that if we know the Student_ID, we can uniquely determine the student's Name.

Similarly:

Student_ID → Department

because each Student_ID corresponds to exactly one department.

For example,

Student_ID → Name means Student_ID determines Name.

Functional dependencies are important in database normalization, especially when designing tables to reduce redundancy and avoid update, insertion, and deletion anomalies.

Types of Functional Dependency

1. Trivial functional dependency
2. Non- trivial dependency
3. Multivalued functional dependency
4. Transitive fuctional dependency
5. Fully functional dependency

Trivial Functional dependency

The dependency of an attribute on a set of attributes is known as trivial fuctional dependency if the set of attribute includes that attribute.

Deterninant A → B Dependent
Dependent B is Subset of A

Emp_id	Name	Age
1	XYZ	54
2	MNO	25
3	ABC	28
4	RST	25
5	ABC	35

Emp_id , Name → Name

Also Emp_id → Emp_id & Name → Name
This are also trivial dependencies too.

Non-Trivial Dependency

If functional dependency A → B holds true where B is not subset of A then this dependency is called non-trivial functional dependency.

Determinant A → B Dependent
Deteminant B is not subset of A

Emp_id	Name	Age
1	XYZ	54
2	MNO	25
3	ABC	28
4	RST	25
5	ABC	35

Employee table

Emp_id, Name → Age

Age is not a subset of Emp_id, Name

Multi Valued Functional Dependency

Multivalued dependency occours when there are more than one independent multivalued In table.

A → (B, C)
so, B → C
C →B

Emp_id	Name	Age
1	XYZ	54
2	MNO	25
3	ABC	28
4	RST	25
5	ABC	35

Emp_id → (Name, Age)

Name ----> Age
Age ----> Name

Transitive Functional dependency

A → B B → C

A → C

Emp_id	Name	Dept_Name	Dept_Building
1	XYZ	R&D	Building A
2	MNO	Design	Building B
3	ABC	Production	Building C
4	RST	Accounts	Building d

Emp_id → Dept_Name Dept_Name → Dept_Building

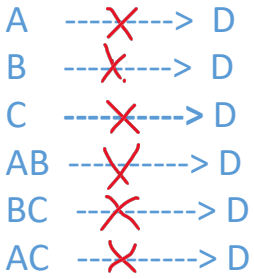
Emp_id → Dept_Building

Fully Functional Dependency

An attribute depends on the entire composite key, and removing any part of the key breaks the dependency

Proper subset

(A, B, C) ----------> D



Student_id	Course _id	Marks
1	A	87
1	B	82
1	C	91
2	A	77
2	B	81
2	C	75

Marks Table

Student_id, course_id -----> Marks
Student_id → Marks
Course_id → Marks

Normalization

Normalization is one of the most important concepts in Advanced Database Management Systems (ADBMS). It is a process of organizing data in database tables to reduce data redundancy and avoid data anomalies.

In simple words: Normalization means breaking a large, poorly designed table into smaller, well-organized tables and connecting them using relationships.

Student_ID	Student_Name	Course_ID	Course_Name	Faculty
101	Amit	C01	DBMS	Dr. Sharma
102	Priya	C01	DBMS	Dr. Sharma
103	Rahul	C02	Java	Dr. Patil
104	Neha	C01	DBMS	Dr. Sharma

Here, **Course_Name** and **Faculty** are repeated for every student taking the same course.

This creates several problems.

Problems

1. Data Redundancy

The same information is stored multiple times.

For example:

`C01 → DBMS → Dr. Sharma` is repeated three times.

2. Update Anomaly

Suppose Dr. Sharma is replaced by Dr. Joshi.

We have to update multiple records.

If one record is missed, the database contains inconsistent information.

3. Insertion Anomaly

Suppose a new course is introduced, but no student has enrolled yet.

We may not be able to store the course information because `Student_ID` is required.

4. Deletion Anomaly

Suppose the only student enrolled in Java leaves the course.

Deleting that student's record may also delete the only information we have about the Java course.

Normalization helps solve these problems.

Main Objectives of Normalization

The Major objectives are:

1. Reduce data redundancy
2. Eliminate data anomalies
3. Improve data consistency
4. Improve database design
5. Make data maintenance easier
6. Ensure proper relationships between tables
7. Reduce unnecessary duplication

Normal Forms

Normalization is generally performed in stages called Normal Forms.

The major normal forms are:

1.1NF – First Normal Form

2.2NF – Second Normal Form

3.3NF – Third Normal Form

4.BCNF – Boyce-Codd Normal Form

5.4NF – Fourth Normal Form

6.5NF – Fifth Normal Form

First normal form:

The relation is in First Normal Form (1NF) if:

- Each cell contains a single/atomic value
- There are no repeating groups
- Each column contains values of the same type
- Each row is uniquely identifiable

Student_ID	Student_Name	Phone
101	Amit	9876, 8765
102	Priya	7654

Example – Not in 1NF because the Phone column contain multiple value. This violates 1NF.

Convert to 1NF

Student_ID	Student_Name	Phone
101	Amit	9876
101	Amit	8765
102	Priya	7654

Now every cell contains a single value.

Therefore, the table satisfies **1NF**.

Second Normal Form – 2NF

A relation is in 2NF if: It is already in 1NF It has no partial dependency

What is Partial Dependency?

Partial dependency occurs when a non-key attribute depends on only part of a composite primary key, instead of depending on the complete primary key.

Consider: Student_Course

Student_ID	Course_ID	Student_Name	Course_Name	Marks
101	C01	Amit	DBMS	85
101	C02	Amit	Java	90
102	C01	Priya	DBMS	78

Suppose the primary key is:

(Student_ID, Course_ID)

This is a **composite key** because it contains two attributes.

Dependencies:

`Student_ID → Student_Name`

`Course_ID → Course_Name`

`Student_ID, Course_ID → Marks`

Notice:

`Student_Name` depends only on `Student_ID not on the complete key(student_id, course_id)`

`Similarly Course_Name` depends only on `Course_ID`.

Therefore, partial dependencies exist.

So the table is **not in 2NF**.

Convert to 2NF

Break the table into three tables.

Student table

Student_ID	Student_Name
101	Amit
102	Priya

Course

Course_ID	Course_Name
C01	DBMS
C02	Java

Student_Course

Student_ID	Course_ID	Marks
101	C01	85
101	C02	90
102	C01	78

Now non-key attributes depend on the complete key of their respective relation.
Therefore, the design is in **2NF**.

Third Normal form 3NF

A relation is in **3NF** if:

1. It is already in **2NF**
2. It has **no transitive dependency**

What is Transitive Dependency?

Suppose:

A → B

and

B → C

Then:

A → C

This is called a **transitive dependency** when B is a non-key attribute.

Student_ID	Student_Name	Dept_ID	Dept_Name
101	Amit	D01	Computer
102	Priya	D02	Information Technology
103	Rahul	D01	Computer

Primary key:
`Student_ID`
Dependencies:
`Student_ID → Student_Name`
`Student_ID → Dept_ID`
`Dept_ID → Dept_Name`
Therefore:
`Student_ID → Dept_ID → Dept_Name`
This is a **transitive dependency**.
So the table is not in 3NF.

Convert to 3NF

Create two tables.

Student

Student_ID	Student_Name	Dept_ID
101	Amit	D01
102	Priya	D02
103	Rahul	D01

Department

Dept_ID	Dept_Name
D01	Computer
D02	Information Technology

Now:
`Student_ID → Student_Name, Dept_ID`
and
`Dept_ID → Dept_Name`
The transitive dependency has been removed. Therefore, the database is in **3NF**.

Important point
3NF mainly deals with transitive dependency.

BCNF – Boyce-Codd Normal Form

BCNF is a stronger version of 3NF.

A relation is in BCNF if:

For every non-trivial functional dependency **X → Y**, X must be a

super key.

In simple language:

Every determinant must be a candidate key/superkey.

Example
Consider:
Student_Course_Faculty

Student	Course	Faculty
Amit	DBMS	Sharma
Priya	DBMS	Sharma
Rahul	Java	Patil

Suppose:

`Student, Course → Faculty`

and

`Faculty → Course`

If `Faculty` is not a candidate key but determines `Course`, the relation may violate BCNF.

The solution is to decompose the relation into smaller relations so that every determinant becomes a candidate key in its relation.

Candidate Key = A possible key that can uniquely identify every record, with no unnecessary attribute.

Example

Consider an `STUDENT` table:

Roll_No	Email	Name	Branch
101	amit@gmail.com	Amit	CSE
102	priya@gmail.com	Priya	IT
103	rahul@gmail.com	Rahul	CSE

Here:

- `Roll_No` uniquely identifies every student.
- `Email` also uniquely identifies every student.
- `Name` may not be unique.

Therefore:

Candidate Keys = {Roll_No, Email}

We can select either `Roll_No` or `Email` as the **Primary Key**.

Consider: STUDENT-Student_ID,Student_Name, Branch

Functional dependency: Student_ID → Student_Name, Branch

Suppose: Student_ID = Candidate Key

Therefore : Student_ID = Super Key So the determinant of the dependency is a super key.

Hence:STUDENT is in BCNF

4NF – Fourth Normal Form

4NF deals mainly with **multivalued dependencies**.

A relation is in 4NF if:

1.It is in BCNF

2.It has no non-trivial multivalued dependency unless the determinant is a superkey.

Example

Suppose a student can have multiple:

•Skills

•Hobbies

Student	Skill	Hobby
Amit	Java	Cricket
Amit	Java	Music
Amit	Python	Cricket
Amit	Python	Music

Skills and hobbies are independent multivalued facts.
This creates unnecessary combinations.
We can decompose into:

Student_skill

Student	Skill
Amit	Java
Amit	Python

Student_Hobby

Student	Hobby
Amit	Cricket
Amit	Music

This removes the multivalued dependency problem.

5NF – Fifth Normal Form

5NF is also called Project-Join Normal Form (PJ/NF)

it deals with join dependencies.

A relation is in 5NF when: It cannot be further decomposed without losing information or creating spurious tuples.

5NF is generally used in more complex database designs.

What is Join Dependency?

To understand 5NF, we first need to understand join

dependency.

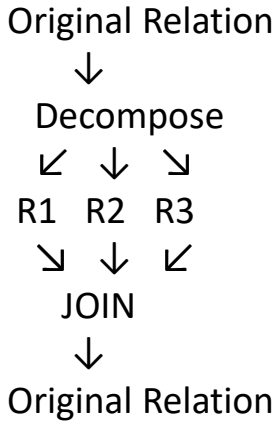
Suppose we have a relation:

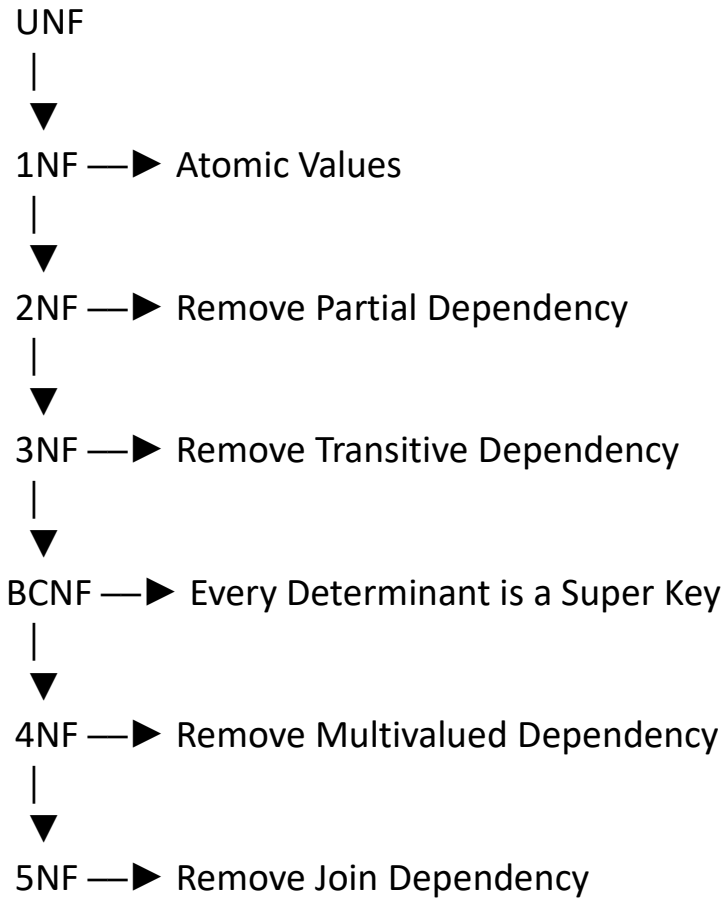
R(A, B, C)

Sometimes the information in this relation can be represented using three smaller relations:

R1(A, B)
R2(B, C)
R3(A, C)

When we can reconstruct the original relation by joining these smaller relations without generating incorrect extra tuples, we have a **join dependency**.




UNF
1NF —► Atomic Values
2NF —► Remove Partial Dependency
3NF —► Remove Transitive Dependency
BCNF —► Every Determinant is a Super Key
4NF —► Remove Multivalued Dependency
5NF —► Remove Join Dependency

Relational Algebra Operations

Operation	Symbol	Purpose
Selection	σ	Selects rows
Projection	π	Selects columns
Join	⋈	Combines relations
Union	∪	Combines tuples
Difference	−	Finds tuples in one relation but not another
Cartesian Product	×	Combines every tuple of two relations
Rename	ρ	Renames a relation

Lossless Join in DBMS

Lossless Join Decomposition means that when we split one relation (table) into two or more smaller relations, we should be able to join them back and get exactly the original table—without losing information and without generating extra/spurious tuples.

Lossless Join is an important concept in Normalization and Decomposition of relations in DBMS. When we decompose a large table into smaller tables, we want to make sure that no information is lost. After decomposition, if we join the smaller tables using their common attributes, we should get exactly the original table.

Simple Definition

Suppose we have a relation:

R(A, B, C)

and we decompose it into:

- **R1(A, B)**
- **R2(B, C)**

If:

R1 ⋈ R2 = R

then the decomposition is called **lossless**.

2. Why is Lossless Join Important?

During normalization, we split a large table into smaller tables to:

•Reduce data redundancy

•Avoid update, insertion and deletion anomalies

•Improve database organization

•Maintain data integrity

•Ensure that the original information can be reconstructed

A good decomposition should not generate **spurious tuples** when the tables are joined.

Why Do We Decompose a Relation?

Consider the following table: Student

Student_ID	Student_Name	Dept_ID	Dept_Name
101	Amit	D01	CSE
102	Rahul	D02	IT
103	Neha	D01	CSE

Here, `Dept_Name` depends on `Dept_ID`.

We can represent the functional dependency as:

$$Dept_ID \rightarrow Dept_Name$$

To reduce redundancy, we can decompose the table.

Student_ID	Student_Name	Dept_ID
101	Amit	D01
102	Rahul	D02
103	Neha	D01

Dept_ID	Dept_Name
D01	CSE
D02	IT

Now the information is stored in two relations.

Can we reconstruct the original STUDENT table from these two tables?

We perform:

$$STUDENT \bowtie DEPARTMENT$$

using the common attribute `Dept_ID`.

We get:

Student_ID	Student_Name	Dept_ID	Dept_Name
101	Amit	D01	CSE
102	Rahul	D02	IT
103	Neha	D01	CSE

This is exactly the original relation.

Therefore, the decomposition is **lossless**.

Detailed Example of Lossless Decomposition

Consider:

R(StudentID, StudentName, DeptID, DeptName)

Functional dependencies:

$$StudentID \rightarrow StudentName, DeptID$$

and

$$DeptID \rightarrow DeptName$$

We decompose R into:

R1(StudentID, StudentName, DeptID)

$R_2(DeptID,\ DeptName)$

Step 1: Find common attribute

$$R_1 \cap R_2 = \{DeptID\}$$

Step 2: Check functional dependency

We know:

$$DeptID \rightarrow DeptName$$

Therefore:

$$DeptID \rightarrow R_2$$

because R_2 contains:

$$DeptID, DeptName$$

So `DeptID` is a **key of R_2**.

Therefore, the decomposition is: Lossless

Dependency Preserving Decomposition

Dependency Preserving Decomposition is an important concept in Database Normalization.

1. Definition

It means that when a relation is divided into two or more smaller relations, all the functional dependencies of the original relation can still be enforced by checking the individual decomposed relations, without having to join them again.

In simple words:

A decomposition is dependency preserving if we can maintain all the original functional dependencies without performing a JOIN operation.

This property is important because it makes it easier to maintain the correctness and consistency of the database.

Suppose we have a relation:

R(A, B, C)

with functional dependencies:

F = { A → B, B → C }

If we decompose it into:

•R1(A, B)

•R2(B, C)

Then:

•In R1, we can check **A → B**

•In R2, we can check **B → C**

So, all dependencies can be checked directly.

Therefore, the decomposition is **dependency preserving decomposition**.

Consider the relation:

STUDENT(Student_ID, Student_Name, Department)

Suppose we have the following functional dependencies:

Student_ID → Student_Name

Student_ID → Department

Therefore:

F = { Student_ID → Student_Name, Student_ID → Department }

Now decompose the relation into:

R_1(Student_ID, Student_Name)

and

R_2(Student_ID, Department)

R1

Student_ID	Student_Name
101	Amit
102	Rahul
103	Neha

The dependency:
Student_ID → Student_Name
can be checked directly in R_1.

R_2

Student_ID	Department
101	CSE
102	IT
103	ECE

The dependency:
Student_ID → Department
can be checked directly in R_2.
Therefore, all the original dependencies can be checked without joining R_1 and R_2.
Hence:
The decomposition is dependency preserving.

Query Processing

What is a Query?

A **query** is a request made by a user to retrieve or manipulate data stored in a database.

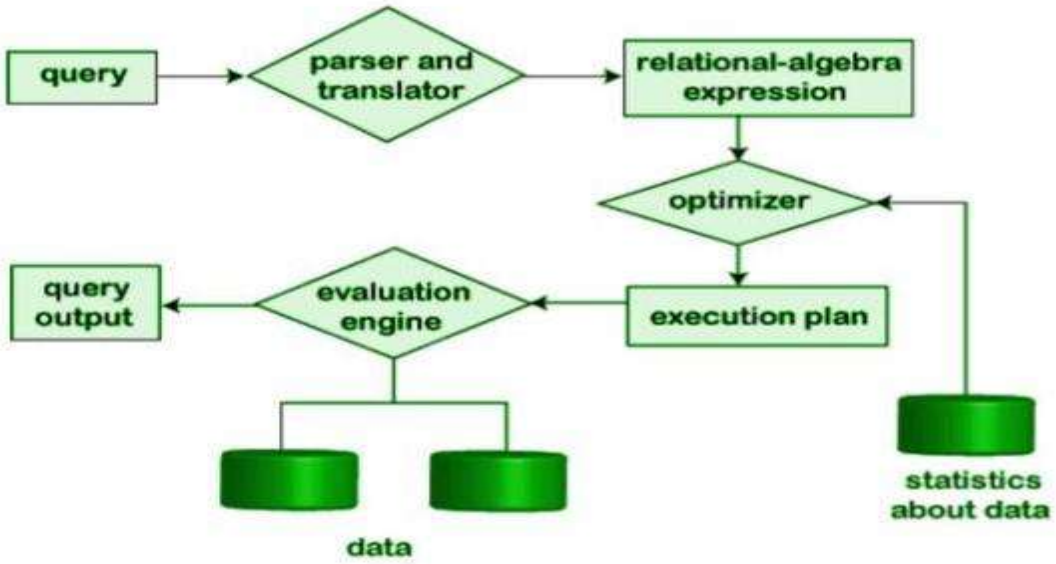
The most commonly used language for writing queries in a relational database is **SQL (Structured Query Language)**.

For example:

```
SELECT Name, Salary
FROM Employee
WHERE Salary > 50000;
```

This query asks the database to find the names and salaries of employees whose salary is greater than ₹50,000.

A query does not directly tell the DBMS **how** to obtain the data. It mainly specifies **what data is required**. The DBMS decides the most efficient way to obtain that data.



Steps in query processing

STEP 1 — Parsing

The first step is **query parsing**.

The DBMS examines the SQL query to determine whether it follows the rules of SQL.

For example:

```
SELECT Name
FROM Employee
WHERE Salary > 50000;
```

The parser checks:

- Is `SELECT` used correctly?
- Does the table `Employee` exist?
- Does the column `Name` exist?
- Does the column `Salary` exist?
- Is the SQL syntax correct?
- Are the data types compatible?

If there is a syntax error, the DBMS reports an error and stops processing the query.

STEP 2 — Query Translation

After parsing and validation, the SQL query is converted into an internal form.

A relational DBMS commonly represents the query using **relational algebra operations**.

For example:

```
SELECT Name
FROM Employee
WHERE Salary > 50000;
```

can be represented as:

```
π Name ( σ Salary > 50000 (Employee) )
```

Where:

- σ = Selection (select row)
- π = Projection (select column)

This representation makes it easier for the optimizer to analyze and improve the query.

Syntax Analysis : Syntax analysis checks the grammatical structure of the SQL statement.

Semantic Analysis: It Varify that the query structure and elements are meaningful and valid

Syntax Analysis in DBMS

Syntax analysis is the first major step in query interpretation. It checks whether an SQL query follows the grammatical and structural rules of the SQL language.

Syntax analysis checks whether the query is written correctly according to SQL grammar.

It does not check whether the table or column actually exists. That is handled later during semantic analysis.

Example

Consider the SQL query:

```
SELECT Name, Salary
FROM Employee
WHERE Salary > 50000;
```

The syntax analyzer checks:

- `SELECT` is used correctly.
- Column names are separated by commas.
- `FROM` is placed after the selected columns.
- A table name follows `FROM`.
- `WHERE` is placed correctly.
- The condition `Salary > 50000` follows valid SQL syntax.
- The statement ends correctly.

Since the query follows SQL grammar, it passes syntax analysis.

Semantic Analysis

After checking the syntax, the DBMS checks the **meaning of the query**.
It verifies things such as:
•Does the `Employee` table exist?
•Does the `Name` column exist?
•Does the `Salary` column exist?
•Is the user authorized to access the table?
•Are the data types compatible?

For example:

```
SELECT StudentName
FROM Employee;
```

If `StudentName` does not exist in the `Employee` table, the query is semantically incorrect even though its syntax is valid.

STEP3 : Relational Algebra is a formal query language used by DBMS to represent and manipulate data stored in relational tables. In query processing, an SQL query is translated into a **relational algebra expression**, which is then optimized and converted into an execution plan.

Example: consider the table: **EMPLOYEE**

Emp_ID	Name	Dept	Salary
101	Amit	IT	60000
102	Ravi	HR	45000
103	Neha	IT	70000

```
SELECT Name
FROM Employee
WHERE Salary > 50000;
```

Its relational algebra expression is:

$$\pi_{Name}\left(\sigma_{Salary>50000}(Employee)\right)$$

Here:

- **σ (Selection)** selects rows.
- **π (Projection)** selects columns.

The DBMS can therefore understand the query as: **first select employees whose salary is greater than 50,000, then display only their names.**

STEP 4 : Query Optimization

Query optimization is the process of finding an efficient execution strategy for a query.

A single SQL query can usually be executed in many different ways.

The DBMS evaluates possible alternatives and selects an execution plan with relatively low estimated cost.

For example, if a table contains **10 lakh records** and an index exists on `Salary`, the DBMS may use the index instead of scanning the entire table.

The objective is not simply to make the SQL statement shorter. The objective is to reduce the resources required to execute it.

Why is Query Optimization Required?
Consider:

```
SELECT *
FROM Employee
WHERE Salary > 50000;
```

Suppose the Employee table contains:

1,000,000 records

There are two possible approaches:

Method 1: Full Table Scan

The DBMS examines every record.

Method 2: Index Scan

If an appropriate index exists on `Salary`, the DBMS may directly locate qualifying records.

Therefore, choosing an appropriate execution strategy can significantly reduce:

- Disk I/O
- CPU operations
- Memory usage
- Execution time

STEP 5 : Query Execution Plan

An **execution plan** is a detailed strategy that specifies how the DBMS will execute a query.
It describes operations such as:

- Which table should be accessed first?
- Should an index be used?
- Which join algorithm should be used?
- In what order should joins be performed?
- Where should filtering occur?
- How should intermediate results be handled?

Example:

```
Index Scan Employee
        ↓
    Selection
        ↓
    Projection
        ↓
      Result
```

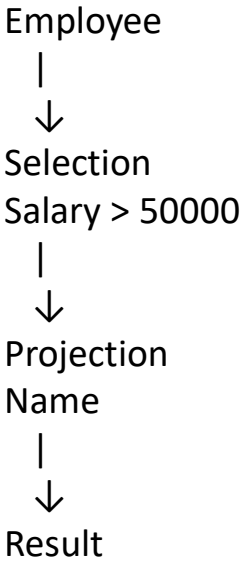
The optimizer chooses the execution plan.

STEP 6: Query Evaluation Plan

A query evaluation plan specifies the sequence of operations used to evaluate a relational algebra expression.

For example:

```
SELECT Name
FROM Employee
WHERE Salary > 50000;
```



Query processing cost estimation

This is the process of estimating the amount of time and system resources required to execute a database query.

Before executing a query, the query optimizer considers different possible execution plans and estimates their costs. It then selects an execution plan that is expected to require fewer resources.

In simple words:

Query cost estimation tells the DBMS: “How expensive will it be to execute this query?”

The cost does not necessarily mean money. It mainly refers to computer resources, especially disk I/O, CPU time, memory usage, and communication cost.

Main Components of Query Processing Cost

The total cost of executing a query can involve several types of costs.

A. Disk I/O Cost

This is generally the **most important cost** in traditional database systems.

It represents the cost of:

- Reading data blocks from disk
- Writing data blocks to disk
- Reading index blocks
- Writing temporary results

For example, if a table occupies 1,000 disk blocks and the DBMS must read all of them, approximately 1,000 block reads are required.

A simplified cost can be written as:

$$\text{Cost} \approx \text{Number of disk block transfers}$$

B. CPU Cost

CPU cost represents the processing required by the processor.

For example, the DBMS may need to:

•Compare values

•Evaluate conditions

•Sort records

•Perform joins

•Calculate aggregate functions

For a condition such as:

```
WHERE Salary > 50000
```

the DBMS may need to compare the salary of many records.

C. Communication Cost

In a distributed database, data may have to travel between different computers or sites.

For example:

The cost of transferring data through the network is called **communication cost**.

It becomes particularly important in distributed database systems.



Index in DBMS

An index in DBMS is a special data structure that is used to find records quickly in a database table. It works somewhat like the index of a textbook. Instead of reading every page to find a topic, we look at the index and directly go to the required page. The index is a type of data structure. It is used to locate and access the data in a database table quickly.

Similarly, instead of searching every record in a table, the DBMS can use an index to locate the required records quickly.

Simple Example

Suppose we have an `Employee` table containing 1,00,000 records.

```
SELECT *
FROM Employee
WHERE Employee_ID = 1050;
```

The main purpose of an index is to **reduce the time required to retrieve data**.

Without an index

The DBMS may need to check many records:

Employee Table
↓
Record 1
Record 2
Record 3
Record 4
...
Record 1050 ← Required record
...
Record 100000

With an index

If an index exists on `Employee_ID`:

Index
↓
Employee_ID = 1050
↓
Data Block Address
↓
Required Record

The DBMS can locate the required record much faster.

How an Index Works(Index structure)

An index generally contains two important pieces of information:

Search Key Value → Pointer to Record (data reference)

Components of an Index

An index generally contains:

1. Search Key

The **search key** is the attribute or set of attributes used to search for records.

Examples:

`Employee_ID`
`Roll_No`
`Name`
`Salary`

For example, if the index is created on `Employee_ID`, then:

`Search Key = Employee_ID`

Pointer(Data reference)

A pointer identifies where the corresponding record or data block can be found.

Conceptually:

```
Search Key          Pointer
-----------         --------
101          →      Block 5
102          →      Block 8
103          →      Block 12
104          →      Block 15
```

The pointer can point to a record or, depending on the index structure, to a block containing the relevant records.

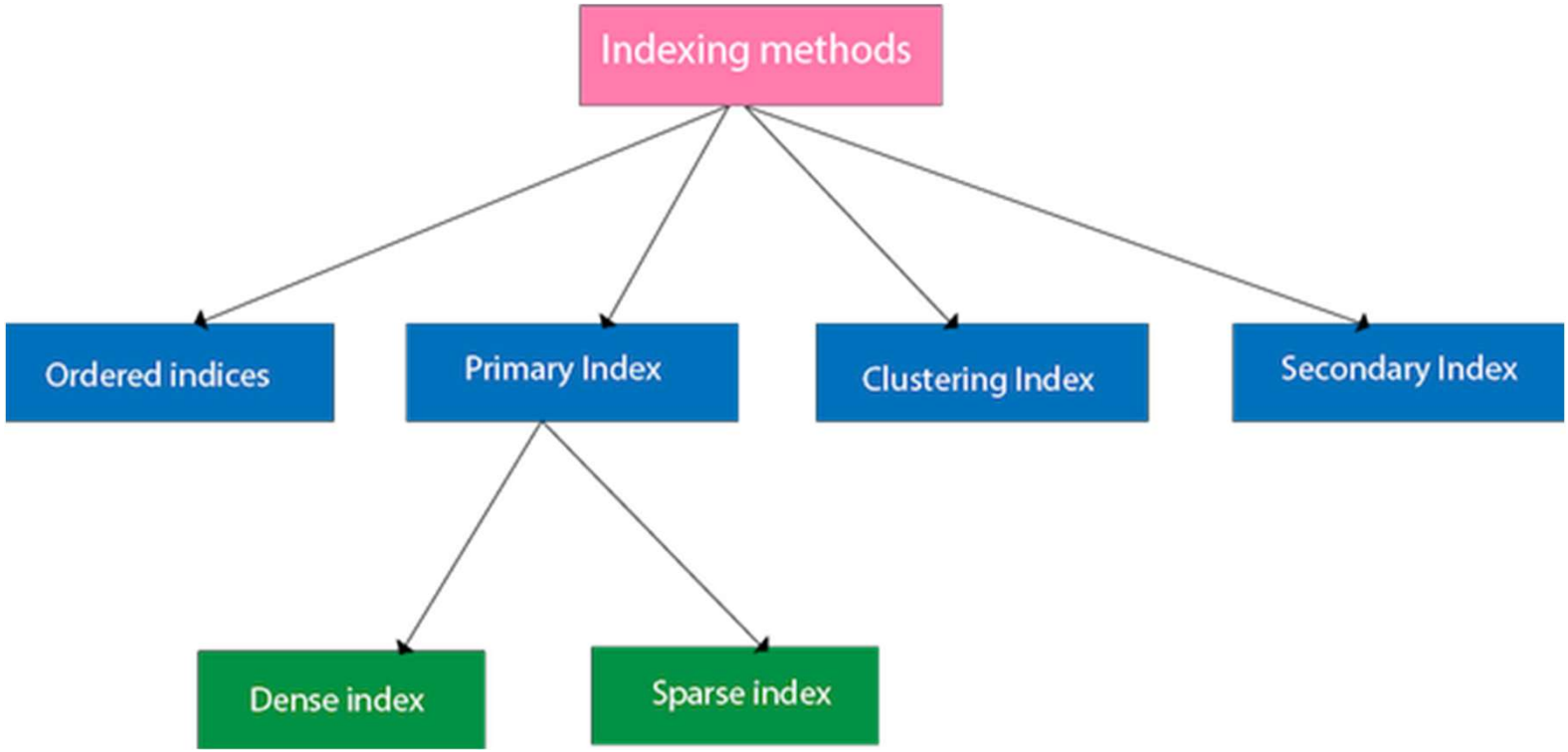
Indexing Methods

Indexing is a technique used in DBMS to speed up the retrieval of records from a database table. Instead of searching the entire table record by record, the DBMS uses an index to quickly locate the required data.

For example, consider a student table:

Roll No	Name	Branch
101	Amit	CSE
102	Rahul	IT
103	Sneha	CSE
104	Priya	ECE

If we want to find the student with Roll No. 103, searching the entire table takes more time when the table contains millions of records. An index can directly point toward the location of the required record.


Indexing methods
Ordered indices
Primary Index
Clustering Index
Secondary Index
Dense index
Sparse index

1) Ordered Indexing

In **ordered indexing**, index entries are maintained in a particular order, usually according to the value of the search key.

For example:

Index
101 → Record 1
102 → Record 2
103 → Record 3
104 → Record 4

Because the entries are ordered, searching can be performed efficiently.

2) Primary Index

A **primary index** is created on the **primary key** of a table.

The primary key is generally unique, and the records in the data file are arranged according to the primary key.

Example

Suppose the data file is arranged according to `Roll_No`.

Example : Student

Roll_no	Names
101	Amit
102	Rahul
103	Sneha
104	Priya
105	Raj

Since the records are arranged according to `Roll_No`, an index can be created on `Roll_No`.

The index may contain:

Primary Index

101 → Block 1
103 → Block 2
105 → Block 3

The index points to the blocks containing the required records. Because of it searching for a record using the primary key becomes faster.

i) Dense Index

In a **dense index**, there is an index entry for **every search-key value or every record**, depending on the indexing organization. Number of record in a table is same as no. of index

Example:

Data:

101 → Amit
102 → Rahul
103 → Sneha
104 → Priya

Advantage

Searching is very fast.

Disadvantage

It requires more storage space.

Dense index:

101 → Record 101
102 → Record 102
103 → Record 103
104 → Record 104

If we want record `102`, the DBMS finds

`101 → Block 1` and then searches inside Block 1.

ii) Sparse Index

In a **sparse index**, index entries are created only for **some search-key values**, commonly one entry per data block. A **sparse index** does not contain an entry for every record.

Example:

Block 1:
101 Amit
102 Rahul
103 Sneha

Block 2:
104 Priya
105 Raj
106 Neha

Sparse index:

101 → Block 1
104 → Block 2

The index points to blocks rather than every individual record.

If we want record `102`, the DBMS finds `101 → Block 1` and then searches inside Block 1.

101 → Block 1
104 → Block 2

Advantage

- Requires less storage
- Smaller index
- Index maintenance can be cheaper

Disadvantage

The DBMS may need additional searching within the block.

3) Clustering Index

A **clustering index** is created when records are physically stored according to a **non-key attribute**, called the clustering field.

For example, suppose students are stored according to their branch:

CSE

Amit
Sneha
Raj

ECE

Priya
Karan

IT

Rahul
Neha

An index can be created on `Branch`:

CSE → Block 1
ECE → Block 2
IT → Block 3

Here, multiple records can have the same value.

Important point

Primary Index → usually based on primary key

Clustering Index → based on an ordering field that may contain duplicate values

Secondary Index

A **secondary index** is an additional index created on a field that is not used for the primary ordering of the data file. It is used to optimize the query processing and access record in database with some info other than primary key.

For example: student

STUDENT

Roll_No	Name	Branch
101	Amit	CSE
102	Rahul	IT
103	Sneha	CSE
104	Priya	ECE

Suppose the table is physically ordered by `Roll_No`, but we frequently search by `Branch`.

We can create: Index on Branch

We get index
CSE → Record 101, Record 103
ECE → Record 104
IT → Record 102

This is a **secondary index**.

JOIN

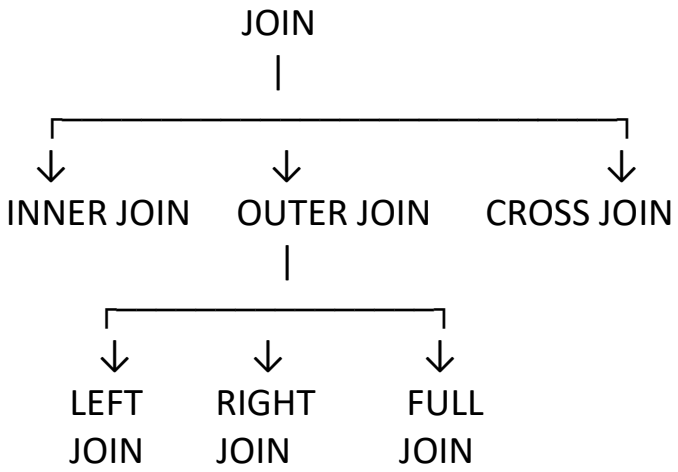
JOIN in SQL is used to **combine rows from two or more tables** based on a related column between them.

In a relational database, information is usually divided into multiple tables. JOIN allows us to retrieve related information from these tables in a single query.

JOIN is an SQL operation used to combine data from multiple tables using a related column or condition.

Types of JOIN

1) Inner Join
2) Left Join
3) Right Join
4) Full Join
5) Cross Join



1) Inner Join

In SQL , INNER JOIN select records that have matching values in both tables as long as the condition is satisfied. It returns the combination of all rows from both the tables where the condition satisfied.

Syntax

```
SELECT table1.column1, table1.column2, table2.column1,....
FROM table1
INNER JOIN table2
ON table1.matching_column = table2.matching_column;
```

Suppose: Student

ID	Name	Dept_ID
101	Amit	10
102	Rahul	20
103	Sneha	10
104	Priya	30

Department

Dept_ID	Dept_Name
10	CSE
20	IT
40	Mechanical

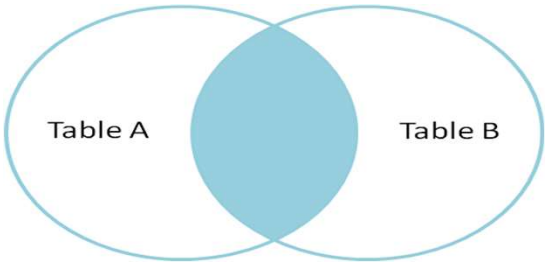
The matching department IDs are:

`10 → CSE`

`20 → IT`

Therefore:

```
SELECT Student.Name, Department.Dept_Name
FROM Student
INNER JOIN Department
ON Student.Dept_ID = Department.Dept_ID;
```



Name	Dept_Name
Amit	CSE
Rahul	IT
Sneha	CSE

`Priya` is not returned because `Dept_ID = 30` does not exist in `DEPARTMENT`.
Similarly, `Mechanical` is not returned because no student has `Dept_ID = 40`.

2. LEFT JOIN

A LEFT JOIN returns:

All rows from the left table + matching rows from the right table.

If there is no match, SQL returns `NULL` for columns from the right table.

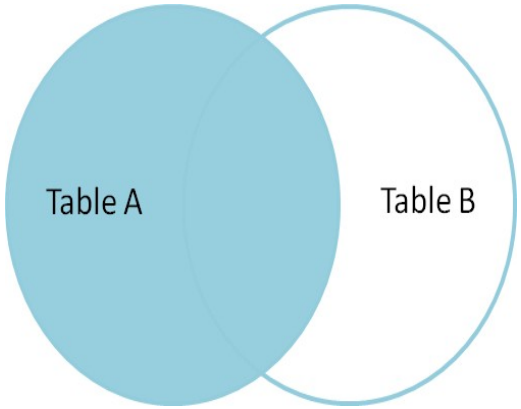
Syntax:

```
SELECT column1, column2, ...
FROM table1
LEFT JOIN table2
ON table1.common_column = table2.matching_column;
```

```
SELECT Student.Name, Department.Dept_Name
FROM Student
LEFT JOIN Department
ON Student.Dept_ID = Department.Dept_ID;
```

Suppose Student contains:

Student_ID	Name	Dept_ID
101	Amit	10
102	Rahul	20
103	Priya	40



But Department contains only:

Dept_ID	Dept_Name
10	CSE
20	IT

Result:

Name	Dept_Name
Amit	CSE
Rahul	IT
Priya	NULL

Priya is still displayed because **LEFT JOIN keeps all records from the left table**.

Why is Priya included?

Because STUDENT is the **left table**, and LEFT JOIN keeps every row from the left table.

Key idea

LEFT JOIN = All left rows + Matching right rows

3) RIGHT JOIN

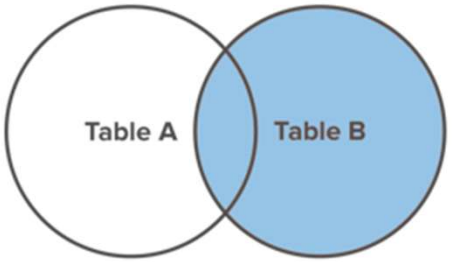
A RIGHT JOIN returns:

All rows from the right table + matching rows from the left table.

Syntax

```
SELECT column1, column2, ...
FROM table1
RIGHT JOIN table2
ON table1.common_column = table2.common_column;
```

```
SELECT Student.Name, Department.Dept_Name
FROM Student
RIGHT JOIN Department
ON Student.Dept_ID = Department.Dept_ID;
```



SQL RIGHT JOIN

Suppose: Student

Student_ID	Name	Dept_ID
101	Amit	10
102	Rahul	20
103	Priya	40

Department has:

Dept_ID	Dept_Name
10	CSE
20	IT
30	ECE

but no student belongs to ECE.

Result:

Name	Dept_Name
Amit	CSE
Rahul	IT
NULL	ECE

ECE is displayed because **RIGHT JOIN keeps all records from the right table**.

Remember:

RIGHT JOIN → Keep everything from RIGHT table.

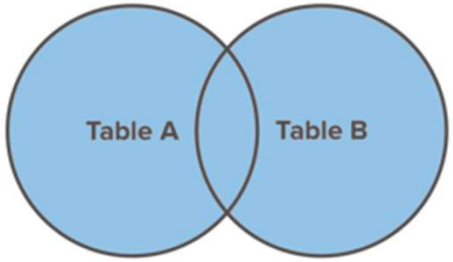
4) FULL OUTER JOIN

A FULL OUTER JOIN returns: All rows from both tables.

It includes:

- Matching rows
- Unmatched rows from the left table
- Unmatched rows from the right table

Where there is no match, `NULL` is displayed.



SQL FULL JOIN

Syntax:

```
SELECT column1, column2, ...
FROM table1
FULL OUTER JOIN table2
ON table1.common_column = table2.common_column;
```

Example:

```
SELECT Student.Name, Department.Dept_Name
FROM Student
FULL OUTER JOIN Department
ON Student.Dept_ID = Department.Dept_ID;
```

Student

Student_ID	Name	Dept_ID
101	Amit	10
102	Rahul	20
103	Priya	30
104	Neha	40

Department

Dept_ID	Dept_Name
10	CSE
20	IT
30	ECE
50	Mechanical

Result:

Student_ID	Name	Dept_ID	Dept_Name
101	Amit	10	CSE
102	Rahul	20	IT
103	Priya	30	ECE
104	Neha	40	NULL
NULL	NULL	50	Mechanical

The 40 record is retained because **FULL JOIN keeps everything from the left table**.

The 50 record is also retained because **FULL JOIN keeps everything from the right table**.

5) CROSS JOIN

A CROSS JOIN is a type of SQL join that combines each row of the first table with every row of the second table. In simple words: CROSS JOIN = Every possible combination of rows from two tables.

It is also called a Cartesian Product.

Syntax :

```
SELECT column1, column2, ...
FROM table1
CROSS JOIN table2;
```

Example:

```
SELECT Student_Name, Course_Name
FROM Student CROSS JOIN Course;
```

Student Table

Student_ID	Student_Name
101	Amit
102	Rahul
103	Priya

There are **3 rows**.

Course Table

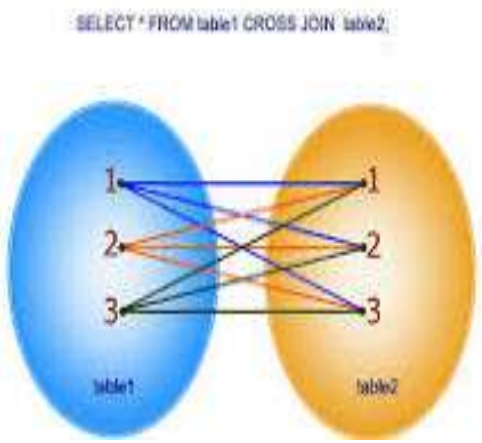
Course_ID	Course_Name
C1	Java
C2	DBMS

There are **2 rows**.

When we perform a CROSS JOIN, **every student is combined with every course**.
Therefore:
Number of rows = 3 × 2 = 6

Result:

Student_Name	Course_Name
Amit	Java
Amit	DBMS
Rahul	Java
Rahul	DBMS
Priya	Java
Priya	DBMS



Join Method Strategies in DBMS

In **query processing**, a **join method strategy** means the technique used by the DBMS to **execute a JOIN between two relations efficiently**.

For example:

```
SELECT *
FROM Student S
JOIN Department D
ON S.Dept_ID = D.Dept_ID;
```

The DBMS has to decide **how to find matching rows** between `Student` and `Department`. This decision is made by the **query optimizer**.

The optimizer can choose one of three basic join strategies when queries require tables to be joined:

- Nested-loop join
- Merge join
- Hash join.

1. Nested Loop Join

This is one of the simplest join strategies.

The DBMS takes one table as the **outer relation** and another as the **inner relation**.

For every row in the outer table, it searches the inner table for matching rows.

Example Suppose: The **Student** table is treated as the outer table and **Department i**s the inner table.

STUDENT

ID	Name	Dept_ID
1	Amit	10
2	Rahul	20
3	Sneha	10

Department

Dept_ID	Dept_Name
10	CSE
20	IT

Conceptually:

For each Student
↓
Search Department
↓
Find matching Dept_ID

For Amit:

```
Amit → Search Department
        ↓
        10 → CSE
```

For Rahul:

```
Rahul → Search Department
         ↓
         20 → IT
```

ID	Name	Dept_ID	Dept_Name
1	Amit	10	CSE
2	Rahul	20	IT
3	Sneha	10	CSE

Advantage

Simple and useful when one relation is small.

Disadvantage

Can be expensive when both tables are large.

Sort - Merge Join

In **Sort-Merge Join**, both tables are first sorted according to the join attribute. Then the sorted tables are scanned together to find matching values.

The basic idea is:
First sort both tables on the join column, and then merge the sorted tables by comparing their values.

Student

Student_ID	Name	Dept_ID
101	Amit	20
102	Rahul	10
103	Priya	30
104	Sneha	20

Department

Dept_ID	Dept_Name
30	CSE
10	IT
20	ECE

Step 1 — Sort Phase
The first step is to sort both relations according to the **join attribute**.
For our example, the join attribute is:
`Dept_ID`

After sorting by `Dept_ID`: IN Student table

Student_ID	Name	Dept_ID
102	Rahul	10
101	Amit	20
104	Sneha	20
103	Priya	30

After sorting: Department

Dept_ID	Dept_Name
10	IT
20	ECE
30	CSE

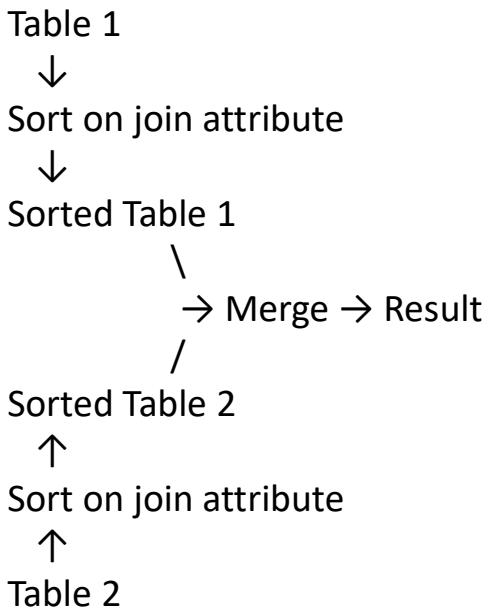
Step 2 — Merge Phase
Now the DBMS compares the sorted tables.

To complete a merge join between tables T1 and T2, the database manager performs the following steps:

1. Read the first row in **Student** . The value for Dept_id is **10**.
2. Scan **Department** until a match (**10**) is found, and then join the two rows.
3. Keep scanning **Department** while the columns match, joining rows.
4. When the **20** in Department is read, go back to **Student** and read the next row.
5. The next value in **Student** is **20**, which matches **Department** so join the rows.
6. Keep scanning **Department** while the columns match, joining rows.
7. When the end of **Department** is reached, go back to **Student** to get the next row. Note that the next value in **Student** is the same as the previous value from **Student**, so **Department** is scanned again, starting at the first 3 in **Department**. The database manager remembers this position.

Then merge them:

10 ↔ 10
20 ↔ 20
30 ↔ 30



Result

Student_ID	Name
102	Rahul
101	Amit
104	Sneha
103	Priya

Dept_ID	Dept_Name
10	IT
20	ECE
20	ECE
30	CSE

Advantage

Very useful for **large relations** and particularly useful when the data is already sorted or sorted access is otherwise beneficial.

Disadvantage

Sorting can be expensive if the tables are not already sorted.

Hash Join

A **Hash Join** uses a **hash function** to divide records into buckets.
It is generally used for **equality joins**, such as:

```
ON Student.Dept_ID = Department.Dept_ID
```

How Hash Join Works

Hash Join has two main phases:

1. Build Phase

2. Probe Phase

Phase 1: Build Phase

Assume `Department` is the smaller table. We create a hash table using `Dept_ID`.

Suppose the hash function produces:

h(10) = 1
h(20) = 2
h(30) = 3

Here:
- **Bucket 1** stores `10`
- **Bucket 2** stores `20`
- **Bucket 3** stores `30`

The **bucket number is obtained from the hash function**.

Phase 2: Probe Phase

In a **hash table**, a **bucket** is a location or storage area where records having the same hash value are stored.

Now scan the `Student` table one row at a time.

Student 101 – Amit

Dept_ID =10
↓
h(10) = 1
↓
Bucket 1
↓
10 → Computer
↓
MATCH

Result:
`Amit → Computer`

Student 102 – Rahul

Dept_ID = 20
↓
h(20) = 2
↓
Bucket 2
↓
20 → IT
↓
MATCH

Result:
`Rahul → IT`

Final Result

Student_Name	Dept_Name
Amit	Computer
Rahul	IT
Priya	Computer
Sneha	Mechanical

Advantages of Hash Function

1.Fast searching – Quickly finds records.
2.Efficient joining – Useful for Hash Join.
3.Less comparison – Searches only the required bucket.
4.Direct access – Hash value gives the bucket location.
5.Good for large data – Efficient for large tables.

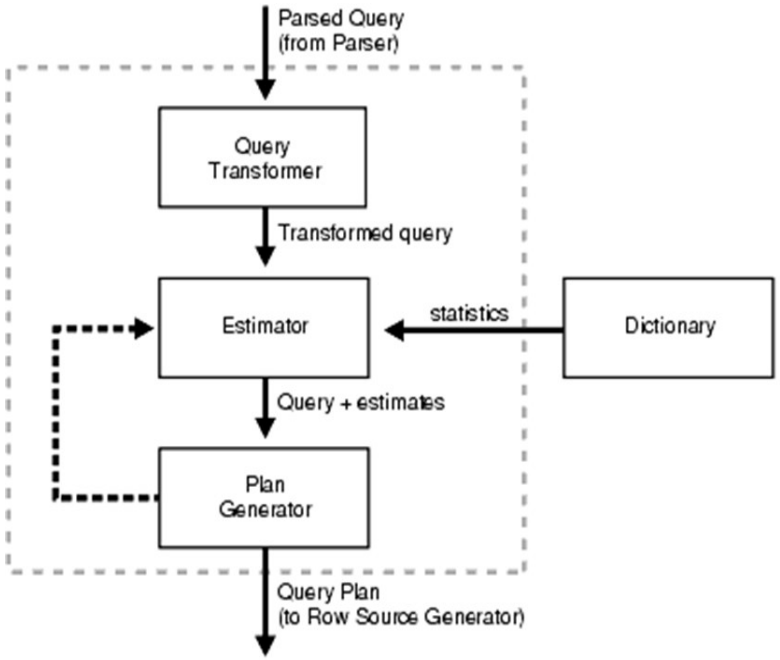
Structure of query optimizer

Query Optimization is a technique of analyzing and deciding an execution plan that computes the result of the query using less number of resources. The main goal of query optimization is to find an execution plan for that query to reduce the time required to process it.

Two main objectives of Query Optimization are:

•Determine the optimal plan to access the database.

•Reduce the time required to execute the query.



The query optimizer operations include:

- Query Transformation
- Estimation
- Plan Generation

Transformer: Each query portion of a statement is query block. It takes parsed query as input which is represented by set of query blocks. It determines that if it is advantageous to change the form of the query to reduce the cost of execution.

The Transformer takes the parsed query and determines if it can be rewritten or restructured into a semantically equivalent form that is cheaper to execute.

• Key Tasks: It performs logical optimizations such as removing redundant conditions, expanding views, pushing down filters (predicates) closer to the data source, and converting subqueries into joins.

Estimator: It determines the over all cost of execution plan. This estimator uses three different measures to determine cost which includes:

Selectivity: It is defined as a fraction of rows from a row set.

Cardinality: It is defined as the number of rows returned by each operation in executed plans.

Cost: It defines the estimated resource consumption for a plan.

To estimate cost, optimizer uses following factors:

- System resources (CPU, Memory and I/O)
- Cardinality
- Size of initial data set

The Plan Generator:

It explores various plans for query block by checking various access paths, join methods and join orders. After checking various paths, optimizer picks the path with the lowest cost.

• Key Tasks: It tests different physical access paths (e.g., using a Full Table Scan vs. an Index Scan) and tries different join algorithms (like Nested Loops, Hash Joins, or Merge Joins) along with different join orderings.